

Objective

The objective of this project is to design and implement a comprehensive big data pipeline that leverages AWS for scalable data storage and visualization, along with a Linux-based PySpark environment for distributed data processing. This project focuses on developing key competencies in data engineering, big data analytics, and cloud-based distributed systems. It emphasizes the efficient handling of large-scale datasets in a cloud environment while addressing critical ethical considerations, such as mitigating bias in training data and safeguarding privacy.

1. Dataset Selection:

This dataset contains information about customers who have applied for a loan or are eligible to apply for one. Each row represents a customer with various attributes such as their checking balance, loan duration, credit history, purpose of loan, loan amount, savings balance, employment duration, income percentage, residence duration, age, housing type, and job details. Additionally, it includes whether the customer has existing loans, their phone status, number of dependents, and whether they defaulted on a loan. The goal of the project might involve analyzing patterns and predicting whether a customer is likely to default on their loan based on these factors.

The "default" column in the dataset indicates whether a loan applicant defaulted on their loan ("yes") or not ("no"). This binary classification helps in analyzing the factors contributing to loan defaults and understanding the risk profile of applicants. By categorizing applicants into "yes" and "no" based on their default status, we can identify patterns in loan behavior and assess the impact of various features like housing type, credit history, and savings account on the likelihood of default.

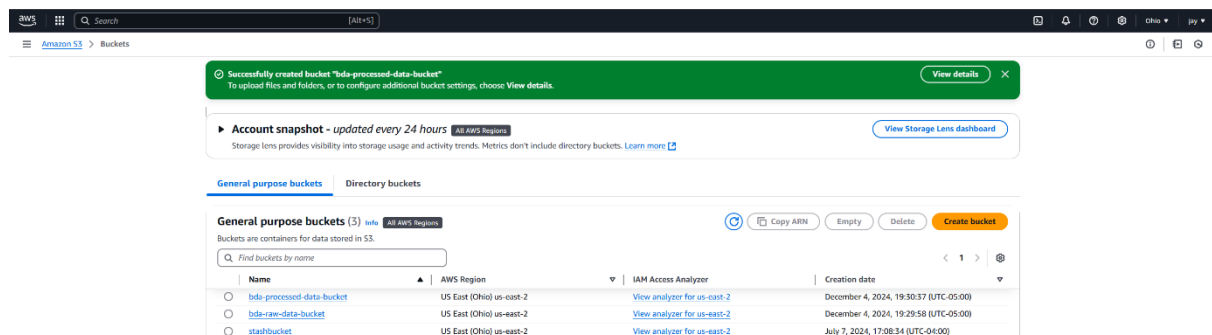
	checking_balance	months_loan_duration	credit_history	purpose	amount	savings_balance	employment_duration	percent_of_income	years_at_residence	age	other_credit	housing	existing_loans_count	job	dependents	phone	default
1	< 0 DM		6 critical	furniture/appliances	1169 unknown		> 7 years		4	4	67 none	own	2 skilled		1 yes		no
2	1- 200 DM		48 good	furniture/appliances	5951 < 100 DM		1- 4 years		2	2	22 none	own	1 skilled		1 no		yes
3	unknown		12 critical	education	2096 < 100 DM		4- 7 years		2	3	49 none	own	1 unskilled		2 no		no
4	< 0 DM		42 good	furniture/appliances	7882 < 100 DM		4- 7 years		2	4	45 none	other	1 skilled		2 no		no
5	< 0 DM		24 poor	car	4870 < 100 DM		1- 4 years		3	4	53 none	other	2 skilled		2 no		yes
6	unknown		36 good	education	9055 unknown		1- 4 years		2	4	35 none	other	1 unskilled		2 yes		no
7	unknown		24 good	furniture/appliances	2835 500 - 1000 DM		> 7 years		3	4	53 none	own	1 skilled		1 no		no
8	1- 200 DM		36 good	car	6948 < 100 DM		1- 4 years		2	2	35 none	rent	1 management		1 yes		no
9	unknown		12 good	furniture/appliances	3059 > 1000 DM		4- 7 years		2	4	61 none	own	1 unskilled		1 no		no
10	1- 200 DM		30 critical	car	5234 < 100 DM		unemployed		4	2	28 none	own	2 management		1 no		yes
11	1- 200 DM		12 good	car	1295 < 100 DM		< 1 year		3	1	25 none	rent	1 skilled		1 no		yes
12	< 0 DM		48 good	business	4308 < 100 DM		< 1 year		3	4	24 none	rent	1 skilled		1 no		yes

2. Environment Setup:

- **AWS S3 for Data Storage:**

Step 1: Create an S3 Bucket to Store Both Raw and Processed Data:

I have created two S3 buckets: *bda_raw_data* and *bda_processed_data*. The *bda_raw_data* bucket is designated for storing raw, unprocessed data, while the *bda_processed_data* bucket will be used for storing data after it has been processed.



Step 2: Upload the Raw Dataset to the S3 Bucket:

I have successfully uploaded the `credit.csv` file to the `bda_raw_data` bucket. This file contains the raw dataset, and it will serve as the starting point for any data processing or analysis steps.

The screenshot shows the Amazon S3 console interface. At the top, the breadcrumb navigation indicates the path: Amazon S3 > Buckets > bda-raw-data-bucket > Upload. The main heading is "Upload" with an "Info" link. Below this, a message states: "Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. [Learn more](#)". A dashed box contains the instruction: "Drag and drop files and folders you want to upload here, or choose **Add files** or **Add folder**." Below this is a section titled "Files and folders (1 total, 90.6 KB)" with a note: "All files and folders in this table will be uploaded." There is a search bar labeled "Find by name" and a table with columns: Name, Folder, Type, Size, and a dropdown arrow. The table contains one entry: "credit.csv" with a folder of "-", type of "text/csv", and size of "90.6 KB". Buttons for "Remove", "Add files", and "Add folder" are present. Below the upload section, the "bda-raw-data-bucket" is selected, showing tabs for Objects, Metadata - Preview, Properties, Permissions, Metrics, Management, and Access Points. The "Objects (1) Info" section is active, showing a list of objects. The table has columns: Name, Type, Last modified, Size, and Storage class. It contains one object: "credit.csv" of type "csv", last modified on "December 6, 2024, 15:57:48 (UTC-05:00)", size "90.6 KB", and storage class "Standard". Above the table are buttons for "Copy S3 URI", "Copy URL", "Download", "Open", "Delete", "Actions", "Create folder", and "Upload".

2. Linux Environment with PySpark:

Step 1: Set up a Linux-based environment, either locally or using an AWS EC2 instance.

Recommendation: Use an AWS EC2 instance for better scalability and AWS integration.

I have set up a Linux-based environment using an AWS EC2 instance named `bda_mini_inst`. This setup ensures better scalability and seamless integration with AWS services, offering flexibility for handling larger datasets and distributed data processing tasks in the project.

The screenshot shows the AWS Management Console. The top section is titled "Instances (1) Info" with a "Last updated 6 minutes ago" status. It includes a search bar "Find Instance by attribute or tag (case-sensitive)" and a dropdown for "All states". Below this is a table with columns: Name, Instance ID, Instance state, Instance type, Status check, Alarm status, Availability Zone, Public IPv4 DNS, Public IPv4 ..., and Elastic IP. The table contains one instance: "bda_mini_inst" with ID "i-088b2c17d11056771", state "Running", type "t2.micro", status check "Initializing", alarm status "View alarms +", availability zone "us-east-1b", public IPv4 DNS "ec2-3-87-32-193.comp...", public IPv4 ... "3.87.32.193", and elastic IP "-". Below the table is a terminal window showing the command prompt and the output of the `df -h` command. The terminal output shows the disk usage for various filesystems, including `/dev/sda1` and `/dev/sdd128`.


```
[ec2-user@ip-172-31-84-19 spark-3.4.0-bin-hadoop3]$ aws s3 ls
2024-12-05 00:30:39 bda-processed-data-bucket
2024-12-05 00:29:59 bda-raw-data-bucket
2024-10-16 12:16:46 stashbucket
[ec2-user@ip-172-31-84-19 spark-3.4.0-bin-hadoop3]$ |
```

3. Data Pipeline Tasks:

Task 1: Data Ingestion from S3:

I used the Boto3 library, providing my AWS access and secret keys, to establish a secure connection with S3. This allowed me to retrieve the required CSV file directly from the S3 bucket for data processing. The retrieved file was used to perform various data transformations and analyses as part of the processing workflow.

```
import boto3

session = boto3.Session(
    aws_access_key_id='',
    aws_secret_access_key='',
    region_name='us-east-2'
)
s3 = session.resource('s3')

# S3 bucket name and the CSV file name:
bucket_name = 'bda-raw-data-bucket'
file_name = 'new_credit.csv'
local_file_path = 'credit.csv'

# The S3 object:
s3_object = s3.Object(bucket_name, file_name)

# Download the file to local storage:
s3_object.download_file(local_file_path)

print("File downloaded successfully!")
```

When the script is run the result can be seen below. The downloaded csv file will be used to perform further processing.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 data_import_script.py
File downloaded successfully!
[ec2-user@ip-172-31-84-19 scripts]$ ls
credit.csv  data_import_script.py
[ec2-user@ip-172-31-84-19 scripts]$
```

The code below demonstrates how to inject the CSV file, downloaded from the S3 bucket, into a Spark session for further processing. It initializes a Spark session and uses the Spark CSV reader to create a DataFrame from the CSV file.

```
from pyspark.sql import SparkSession

# Initializing Spark session:
spark = SparkSession.builder.appName("ReadCSV").getOrCreate()

# Reading CSV file into DataFrame:
df = spark.read.csv("credit.csv", header=True, inferSchema=True)

# Displaying the first 5 rows:
df.show(5)
```

The image below shows the top 5 rows of the data imported from the csv.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 data_injecting.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 03:38:34 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

	checking_balance	months_loan_duration	credit_history	purpose	amount	savings_balance	employment_duration	percent_of_income	years_at_residence	age	other_credit	housing	existing_loans_count	job
1	< 0 DM	6.0	critical	furniture/appliances	1169.0	unknown	> 7 years	4.0	4.0	67.0	none	own	2.0	skilled
2	1 - 200 DM	48.0	good	furniture/appliances	5951.0	< 100 DM	1 - 4 years	2.0	2.0	22.0	none	own	1.0	skilled
3	unknown	12.0	critical	education	2096.0	< 100 DM	4 - 7 years	2.0	3.0	49.0	none	own	1.0	unskilled
4	< 0 DM	42.0	good	furniture/appliances	7882.0	< 100 DM	4 - 7 years	2.0	4.0	45.0	none	other	1.0	skilled
5	< 0 DM	24.0	poor	car	4870.0	< 100 DM	null	3.0	4.0	null	none	other	2.0	skilled

only showing top 5 rows

outliers, performing necessary transformations, and applying aggregation where required.

- Before performing any processing on the imported dataset, it is essential to understand the data types of the columns and gather information about the dataset.

In the image below, we can see the schema of the dataset, which provides the data types of each column, as well as the summary statistics for the numerical columns, including count, mean, standard deviation, min, and max values.

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 03:54:35 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
root
 |-- checking_balance: string (nullable = true)
 |-- months_loan_duration: double (nullable = true)
 |-- credit_history: string (nullable = true)
 |-- purpose: string (nullable = true)
 |-- amount: double (nullable = true)
 |-- savings_balance: string (nullable = true)
 |-- employment_duration: string (nullable = true)
 |-- percent_of_income: double (nullable = true)
 |-- years_at_residence: double (nullable = true)
 |-- age: double (nullable = true)
 |-- other_credit: string (nullable = true)
 |-- housing: string (nullable = true)
 |-- existing_loans_count: double (nullable = true)
 |-- job: string (nullable = true)
 |-- dependents: double (nullable = true)
 |-- phone: string (nullable = true)
 |-- default: string (nullable = true)

24/12/08 03:54:48 WARN package: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
```

	checking_balance	months_loan_duration	credit_history	purpose	amount	savings_balance	employment_duration	percent_of_income	years_at_residence	age	other_credit	housing	existing_loans_count	job
count	980	979	979	979	979	979	979	979	979	979	979	979	979	979
mean	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173	20.914198161389173
stddev	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948	1.1548717940717948
min	1 - 200 DM	4.0	critical	business	250.0	100 - 500 DM	1 - 4 years	1.0	1.0	19.0	bank	other	1.0	unskilled
max	10management	1.0	no	no	18424.0	unknown	unemployed	4.0	4.0	75.0	store	rent	2.0	skilled

Data Transformation: To start with data transformation, it is crucial to first identify any missing values or outliers to ensure the data is clean and consistent. This step helps in detecting inconsistencies that could affect the accuracy of further analysis. Handling missing values and outliers appropriately is essential to improve the quality of the dataset and make it suitable for meaningful transformations and modeling.

Running the script to identify missing values in each column reveals the following count of missing values for each column, as shown in the results below.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 missing_values.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 14:00:29 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

	checking_balance	months_loan_duration	credit_history	purpose	amount	savings_balance	employment_duration	percent_of_income	years_at_residence	age	other_credit	housing	existing_loans_count	job	dependents	phone	default
count	26	21	28	26	23	27	22	30	29	27	28	26	20	27	25	29	29

[ec2-user@ip-172-31-84-19 scripts]\$

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, mean

# Initialize Spark session
spark = SparkSession.builder.appName("Handle Missing Values").getOrCreate()

# Load dataset
df = spark.read.csv("credit.csv", header=True, inferSchema=True)

# Handle missing values for numerical columns
numerical_columns = [col_name for col_name, dtype in df.dtypes if dtype in ('int', 'double')]
for col_name in numerical_columns:
    mean_value = df.select(mean(col(col_name))).collect()[0][0]
    df = df.fillna({col_name: mean_value})

# Handle missing values for categorical columns
categorical_columns = [col_name for col_name, dtype in df.dtypes if dtype == 'string']
for col_name in categorical_columns:
    mode_value = df.groupBy(col_name).count().orderBy('count', ascending=False).first()[0]
    df = df.fillna({col_name: mode_value})

# Save cleaned DataFrame to CSV
df.write.csv('cleaned_credit.csv', header=True, mode='overwrite')

spark.stop()

```

The code above handles missing values in the dataset as follows:

Numerical columns: Missing values are replaced with the mean (average) of the entire column.

Categorical columns: Missing values are replaced with the most frequent value (mode) in the column.

We chose to impute missing data instead of removing it to preserve valuable information and maintain dataset consistency. Removing rows could lead to data loss and potential bias, while imputation ensures that the dataset remains complete and balanced, allowing for more reliable analysis.

The new cleaned_credit.csv file is created that is free of any missing or null values. The same can be seen in the image below.

```

[ec2-user@ip-172-31-84-19 scripts]$ ls
cleaned_credit.csv  credit.csv  data_import_script.py  data_injecting.py  desc_data.py  missing_values.py  process_missing_values.py
[ec2-user@ip-172-31-84-19 scripts]$

```

When the missing values script is run again on the cleaned_credit.csv we can see that it no more contains any missing values

```

[ec2-user@ip-172-31-84-19 scripts]$ python3 missing_values.py
Setting default log level to "WARN".
To adjust logging level use setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 16:46:41 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|checking_balance|months_loan_duration|credit_history|purpose|amount|savings_balance|employment_duration|percent_of_income|years_at_residence|age|other_credit|housing|existing_loans_count|job|dependents|phone|del|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Upon analyzing the data, it was determined that there were no significant outliers that could potentially skew or affect the analysis. The distribution of values across columns appeared normal, and no extreme values were found that could introduce bias or distort the results. Therefore, outlier removal was not necessary for this dataset.

The "months_loan_duration" column spans a wide range of values, and transforming it into a categorical variable can provide more meaningful insights. By classifying the loan durations into categories such as short-term, medium-term, and long-term, we can facilitate more effective analysis and make it easier to apply classification models down the line. This transformation helps to reduce complexity and highlight key patterns in the data.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import when
import pandas as pd

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("cleaned_credit.csv", header=True, inferSchema=True)

# Creating Loan Duration Category column:
df = df.withColumn(
    "loan_duration_category",
    when(df["months_loan_duration"] <= 12, "Short-Term")
    .when((df["months_loan_duration"] > 12) & (df["months_loan_duration"] <= 36), "Medium-Term")
    .otherwise("Long-Term")
)
new_df = df.toPandas()

# Updated dataframe to a new CSV:
new_df.to_csv('credit_with_loan_duration_category.csv', index=False)

spark.stop()
```

The code above demonstrates how to categorize the "months_loan_duration" column into three distinct categories: **Short-Term**, **Medium-Term**, and **Long-Term**. If the loan duration is 12 months or less, it is classified as **Short-Term**. Loan durations between 13 and 36 months are labeled as **Medium-Term**, and any duration greater than 36 months is categorized as **Long-Term**.

When the script is executed to categorize the months_loan_duration, the results are displayed as shown below.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 loan_term_intro.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 18:05:27 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
[ec2-user@ip-172-31-84-19 scripts]$ ls
cleaned_credit.csv  credit.csv  credit_with_loan_duration_category.csv  data_import_script.py  data_injecting.py  desc_data.py  loan_term_intro.py  missing_values.py  process_missing_values.py
[ec2-user@ip-172-31-84-19 scripts]$
```

All other columns are already in the correct format and ready for direct analysis.

Data Aggregation:

1. Average Loan Amount by Loan Duration Category:

- Aggregating the loan amounts by the new **loan_duration_category** column to calculate the average loan amount for **Short-Term**, **Medium-Term**, and **Long-Term** loans.

```
from pyspark.sql import functions as F
from pyspark.sql import SparkSession

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Aggregating average loan amount by loan duration category:
avg_loan_by_duration = df.groupBy("loan_duration_category").agg(
    F.avg("amount").alias("avg_loan_amount")
)

# Results:
avg_loan_by_duration.show()
```


From the code above which loads the dataset and calculates the average loan amount for each loan duration category using PySpark. It groups the data by "loan_duration_category" and displays the results.

The results show that long-term loans receive the highest average amounts, followed by medium-term loans. Short-term loans, on the other hand, have the lowest average loan amounts. This suggests that lenders are more likely to offer larger loans for longer durations and smaller amounts for shorter durations.

```
(ec2-user@ip-172-31-84-19 scripts)$ python3 avg_loan_amt_loan_dur.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 18:21:16 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+
|loan_duration_category|  avg_loan amount|
+-----+-----+
|      Long-Term      | 7645.770114942528|
|    Medium-Term      | 3530.2671480144404|
|    Short-Term       | 1811.442896935933|
+-----+-----+

(ec2-user@ip-172-31-84-19 scripts)$
```

2. Total Number of Loans by Employment Duration:

- Grouping the data by **employment_duration** and count the total number of loans in each category to understand how employment stability relates to loan issuance.

```
from pyspark.sql import functions as F
from pyspark.sql import SparkSession

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Aggregating total number of loans by employment duration:
total_loans_by_employment = df.groupBy("employment_duration").agg(
    F.count("months_loan_duration").alias("total_loans")
)

# Results:
total_loans_by_employment.show()
```

The code loads a dataset and calculates the total number of loans for each employment duration category using PySpark. It groups the data by the "employment_duration" column and uses the count() function to determine the total number of loans in each group. The results are then displayed, showing the number of loans for each employment duration category.

The results show the total number of loans distributed across different employment durations. The highest number of loans (339) is given to individuals with 1 to 4 years of employment, followed by those with more than 7 years of employment (253 loans). Individuals with 4 to 7 years of employment receive 174 loans, while those employed for less than a year get 172 loans. The fewest loans (62) are given to unemployed individuals, indicating a lower likelihood of loan approval for those without employment.

```
(ec2-user@ip-172-31-84-19 scripts)$ python3 totl_loans_empl_dur.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 18:30:48 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+
|employment_duration|total_loans|
+-----+-----+
|    4 - 7 years    |      174  |
|    1 - 4 years    |      339  |
|    > 7 years      |      253  |
|    unemployed     |        62  |
|    < 1 year       |      172  |
+-----+-----+

(ec2-user@ip-172-31-84-19 scripts)$
```


3. Average Age of Customers by Housing Type:

- Grouping the data by housing (e.g., own vs. other) and calculating the average age of customers to see if there's a correlation between housing type and customer age.

```
from pyspark.sql import functions as F
from pyspark.sql import SparkSession

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Aggregating average age by housing type:
avg_age_by_housing = df.groupBy("housing").agg(
    F.avg("age").alias("avg_age")
)

# Results:
avg_age_by_housing.show()
```

The code loads a dataset and calculates the average age for each housing category using PySpark. It groups the data by the "housing" column and uses the `avg()` function to compute the average age for each group. The results are then displayed, showing the average age for individuals in different housing situations.

The results show the average age of individuals based on their housing status. Those who own their homes have the highest average age at approximately 35.6 years, followed by individuals in "other" housing situations with an average age of about 43.8 years. Renters have the lowest average age at around 30.4 years, suggesting that younger individuals are more likely to rent, while homeownership tends to be associated with slightly older individuals.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 avg_age_hous_type.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 18:37:41 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
+-----+-----+
|housing|    avg_age|
+-----+-----+
|   own | 35.59326788218794|
|  other | 43.81481481481482|
|   rent | 30.368715083798882|
+-----+-----+
[ec2-user@ip-172-31-84-19 scripts]$
```

4. Loan Approval Rate by Job Type:

- Calculate the percentage of loan approvals by **job** type to understand which job categories have higher approval rates.

```
from pyspark.sql import functions as F
from pyspark.sql import SparkSession

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Calculating loan approval rate by job type:
loan_approval_by_job = df.groupBy("job").agg(
    F.avg((df["default"] == "yes").cast("int")).alias("approval_rate")
)

# Results:
loan_approval_by_job.show()
```

The code loads a dataset and calculates the loan approval rate for each job category using PySpark. It groups the data by the "job" column and calculates the approval rate for each group. This is done by

dividing the number of approved loans by the total number of loans in each job category. The results are then displayed, showing the approval rates for different job types.

The results show the loan approval rate for individuals based on their job type. Management professionals have the highest approval rate at approximately 34.5%, followed by unemployed individuals with a rate of 31.8%. Skilled workers have a slightly lower approval rate of around 29.5%, and unskilled workers have the lowest approval rate at 28%. This suggests that job type influences the likelihood of loan approval, with management positions having the highest approval rates.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 loan_approval_by_job.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 18:42:27 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

job	approval_rate
management	0.34459459459459457
unskilled	0.28
skilled	0.29523809523809524
unemployed	0.3181818181818182

```
[ec2-user@ip-172-31-84-19 scripts]$
```

5. Loan Amount by Credit History:

- Aggregate the **amount** of loans by **credit_history** to identify trends in loan amounts based on the credit history classification.

```
from pyspark.sql import functions as F
from pyspark.sql import SparkSession

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Aggregating total loan amount by credit history:
loan_by_credit_history = df.groupBy("credit_history").agg(
    F.sum("amount").alias("total_loan_amount")
)

# Results:
loan_by_credit_history.show()
```

The results show the total loan amount distributed across different credit history categories. Individuals with a "good" credit history have the highest total loan amount, totaling approximately 1.61 million. Those with a "critical" credit history have the second-highest amount at around 904,795. "Perfect" credit history individuals have a total loan amount of 212,227, while "very good" and "poor" credit histories account for 163,899 and 378,629, respectively. This indicates that better credit histories tend to receive higher loan amounts.

The code loads a dataset and calculates the total loan amount for each credit history category using PySpark. It groups the data by the "credit_history" column and uses the `sum()` function to compute the total loan amount for each group. The results are then displayed, showing the total loan amounts for different credit history categories.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 loan_amt_on_cred_hist.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 18:48:45 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

credit_history	total_loan_amount
very good	163899
critical	904795
perfect	212227
poor	378629
good	1611708

```
[ec2-user@ip-172-31-84-19 scripts]$
```

Task 3: Store Processed Data Back to S3:

The code uploads the file from local EC2 instance to bda-processed-data-bucket S3 bucket using AWS boto3. It specifies the local file path on the instance and the destination S3 bucket, then transfers the file securely to the cloud storage for further use.

```
import boto3

# Initializing the session:
session = boto3.Session(
    aws_access_key_id='',
    aws_secret_access_key='',
    region_name='us-east-1'
)
s3 = session.resource('s3')

# S3 bucket name and the file name:
bucket_name = 'bda-processed-data-bucket'
file_name = 'credit_with_loan_duration_category.csv'
local_file_path = 'credit_with_loan_duration_category.csv'

# The S3 object:
s3_object = s3.Object(bucket_name, file_name)

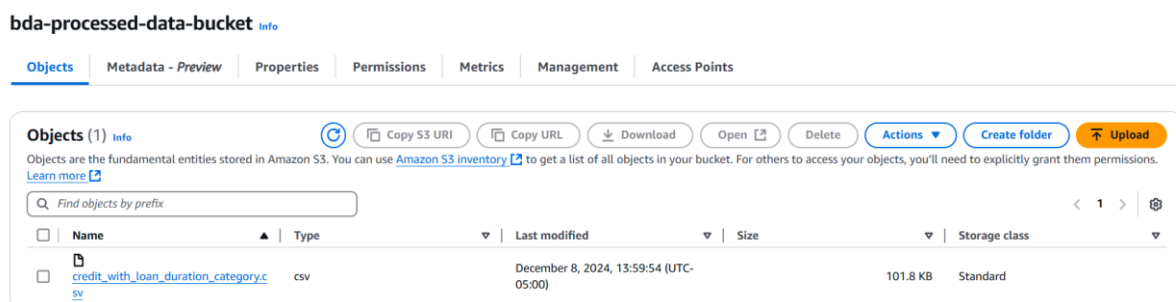
# Upload the file to S3:
s3_object.upload_file(local_file_path)

print("File uploaded successfully!")
```

When the script runs successfully, it returns a message confirming that the file has been successfully uploaded to the destination bucket.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 upload_to_s3.py
File uploaded successfully!
[ec2-user@ip-172-31-84-19 scripts]$
```

From the image below we can confirm that the file has been uploaded successfully to the destination bucket:



Task 4: Data Analysis Using Spark SQL:

1. Retrieving top purpose to avail a loan:

By performing this analysis, we will gain insights into the most common reasons users apply for loans, based on the loan amounts they request or have received. This will help identify trends in loan purposes, highlighting the areas where people are most likely to seek financial assistance.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import when
import pandas as pd

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Registering DataFrame as a SQL temporary view:
df.createOrReplaceTempView("loan_data")

# SQL query to get top purposes based on loan amount:
top_purposes_query = """
SELECT purpose, SUM(amount) as total_amount
FROM loan_data
GROUP BY purpose
ORDER BY total_amount DESC
LIMIT 4
"""

# Executing the query:
top_purposes = spark.sql(top_purposes_query)

# Results:
top_purposes.show()
```

This code initializes a Spark session and loads a dataset into a DataFrame. It then registers the DataFrame as a temporary SQL view, allowing SQL queries to be executed on the data. The code runs a query to retrieve the top 4 loan purposes based on the total loan amount. It groups the data by the "purpose" column, calculates the sum of loan amounts for each purpose, and orders the results in descending order of the total amount. Finally, the results are displayed, showing the top loan purposes.

The results show the total loan amount allocated for different purposes. The highest loan amounts are for purchasing a car (1,269,881) and for furniture or appliances (1,269,644). Business loans total 403,330, while education loans account for the lowest amount at 169,873. This indicates that loans for car purchases and furniture/appliances are the most common, while education loans receive comparatively smaller amounts.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 top_purpose.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 19:28:36 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
+-----+-----+
| purpose | total_amount |
+-----+-----+
| car | 1269881 |
| furniture/appliances | 1269644 |
| business | 403330 |
| education | 169873 |
+-----+-----+
[ec2-user@ip-172-31-84-19 scripts]$
```

2. Loan Duration vs. Age of Borrowers along with min and max age in each category:

Deriving insights on how the loan duration ("short-term", "medium-term", "long-term") correlates with the age of the borrower. This could highlight trends about which age groups tend to opt for longer or shorter loan durations.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import when
import pandas as pd

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Registering DataFrame as a SQL temporary view:
df.createOrReplaceTempView("loan_data")

# SQL query to analyze loan duration vs. age:
loan_duration_age_query = """
SELECT
    loan_duration_category,
    AVG(age) as average_age
FROM loan_data
GROUP BY loan_duration_category
ORDER BY average_age
"""

# Executing the query:
loan_duration_age_analysis = spark.sql(loan_duration_age_query)

# Results:
loan_duration_age_analysis.show()

# SQL query to analyze loan duration vs. age with min and max age:
loan_duration_age_min_max_query = """
SELECT
    loan_duration_category,
    MIN(age) as min_age,
    MAX(age) as max_age
FROM loan_data
GROUP BY loan_duration_category
ORDER BY loan_duration_category
"""
```

The first query analyzes the relationship between loan duration categories and the average age of borrowers, grouping the data by loan duration and ordering the results by average age. The second query further explores the loan duration categories by calculating the minimum and maximum ages within each category. Both queries are executed using Spark SQL, and the results are displayed to provide insights into how age varies across different loan durations.

The results show that short-term loan borrowers have the highest average age at 36.7 years, followed by long-term (36.0 years) and medium-term borrowers (34.7 years). In terms of age range, short-term loans have the widest span, with borrowers aged 19 to 75, while medium-term loans range from 20 to 75 years, and long-term loans from 21 to 65 years. This suggests that short-term loans attract younger borrowers, while long-term loans tend to have slightly older borrowers.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 loan_dur_vs_age.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 19:41:00 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+
|loan_duration_category|average_age|
+-----+-----+
|Medium-Term|34.7057761732852|
|Long-Term|36.0|
|Short-Term|36.73259052924791|
+-----+-----+

+-----+-----+-----+
|loan_duration_category|min_age|max_age|
+-----+-----+-----+
|Long-Term|21|65|
|Medium-Term|20|75|
|Short-Term|19|75|
+-----+-----+-----+

[ec2-user@ip-172-31-84-19 scripts]$
```

3. Loan Amount Distribution by Credit History:

Relationship between loan amounts and credit history status. This analysis could help assess whether people with better credit histories are granted larger loans.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import when
import pandas as pd

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Registering DataFrame as a SQL temporary view:
df.createOrReplaceTempView("loan_data")

# SQL query to analyze loan amount distribution by credit history:
loan_amount_credit_history_query = """
SELECT
    credit_history,
    AVG(amount) as avg_loan_amount,
    MIN(amount) as min_loan_amount,
    MAX(amount) as max_loan_amount
FROM loan_data
GROUP BY credit_history
ORDER BY avg_loan_amount DESC
"""

# Executing the query:
loan_amount_credit_history_analysis = spark.sql(loan_amount_credit_history_query)

# Results:
loan_amount_credit_history_analysis.show()
```

Runs a SQL query to analyze the distribution of loan amounts based on credit history. The query calculates the average, minimum, and maximum loan amounts for each credit history category, excluding null values. The results are ordered by the average loan amount in descending order, providing insights into how loan amounts vary across different credit history groups.

The results show the loan amount distribution across different credit history categories. Borrowers with a "perfect" credit history have the highest average loan amount at 5,564, with loan amounts ranging from 950 to 18,424. "Poor" credit history borrowers have the second-highest average of 4,344, with loan amounts between 585 and 15,653. The other categories—"very good," "critical," and "good"—show decreasing average loan amounts, with a similar range of loan values. This suggests that better credit histories are associated with higher average loan amounts.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 loan_amt_distr.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 19:57:06 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
```

credit_history	avg_loan_amount	min_loan_amount	max_loan_amount
perfect	5564.351351351352	950.0	18424.0
poor	4344.134146341464	585.0	15653.0
very good	3250.6170212765956	339.0	14782.0
critical	3084.7659574468084	250.0	14179.0
good	3063.4143426294822	276.0	15857.0

```
[ec2-user@ip-172-31-84-19 scripts]$
```

4. Percentage Distribution of Loan Applicants by Housing Type:

This analysis calculates the percentage distribution of loan applicants based on their housing type: own, rent, or other. It provides insights into how different housing situations are represented among the applicants, which can help identify trends or biases in loan applications.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import when
import pandas as pd

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Registering DataFrame as a SQL temporary view:
df.createOrReplaceTempView("loan_data")

# SQL query to calculate percentage of people with different housing types:
housing_percentage_query = """
SELECT
    housing,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM loan_data), 2) AS housing_percentage
FROM loan_data
GROUP BY housing
"""

# Executing the query:
housing_percentage_analysis = spark.sql(housing_percentage_query)

# Results:
housing_percentage_analysis.show()
```

SQL query to calculate the percentage of people with different housing types in the dataset. The query counts the number of records for each housing type and divides it by the total number of records to compute the percentage. The results are displayed with the housing type and the corresponding percentage, providing insights into the distribution of housing types among the borrowers.

The results show the percentage of borrowers with different housing types. The majority (71.3%) own their homes, while 17.9% rent, and 10.8% fall under the "other" category. This distribution highlights that most borrowers have ownership of their housing, with a smaller proportion renting or having other housing situations.

```
[ec2-user@ip-172-31-84-19 scripts]$ python3 appl_hous_typ_per.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 20:16:49 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
+-----+-----+
|housing|housing_percentage|
+-----+-----+
|   own|             71.30|
| other|             10.80|
|   rent|             17.90|
+-----+-----+

[ec2-user@ip-172-31-84-19 scripts]$
```


5. Employment Duration and Loan Approval Insight:

This analysis tries to answer How does the length of employment affect the loan approval process or loan amount?

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import when
import pandas as pd

# Initializing Spark session:
spark = SparkSession.builder.appName("Loan Duration Category").getOrCreate()

# Loading dataset:
df = spark.read.csv("credit_with_loan_duration_category.csv", header=True, inferSchema=True)

# Registering DataFrame as a SQL temporary view:
df.createOrReplaceTempView("loan_data")

# SQL query to calculate the average loan amount and default rate based on employment duration:
employment_duration_analysis_query = """
SELECT
    employment_duration,
    AVG(amount) AS avg_loan_amount,
    ROUND(SUM(CASE WHEN default = 'yes' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS default_rate
FROM loan_data
GROUP BY employment_duration
ORDER BY employment_duration
"""

# Executing the query:
employment_duration_analysis = spark.sql(employment_duration_analysis_query)

# Results:
employment_duration_analysis.show()
```

This code performs an analysis of loan amounts and default rates based on employment duration. It initializes a Spark session and loads the dataset from a CSV file. The dataset is registered as a temporary SQL view, allowing SQL queries to be run on it. The SQL query calculates the average loan amount (avg_loan_amount) and the default rate (default_rate) for each employment duration category. The default rate is calculated by counting the number of defaults and dividing it by the total number of records in that category. Finally, the query groups the results by employment duration and orders them for better readability. The results are displayed using the show() function.

The results show the average loan amount and default rate for borrowers based on their employment duration. Borrowers with 1-4 years of employment have an average loan amount of 3,125, with a default rate of 30.68%. Those employed for 4-7 years have a slightly higher average loan amount (3,602) and a lower default rate (22.41%). Borrowers with less than 1 year of employment have the lowest average loan amount (2,952) and the highest default rate (40.70%). Unemployed borrowers have the highest average loan amount (4,217) but also experience a high default rate of 37.10%. These insights indicate that loan amounts and default rates vary significantly based on employment duration.

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
24/12/08 20:53:22 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

+-----+-----+-----+
|employment_duration|avg_loan_amount|default_rate|
+-----+-----+-----+
|1 - 4 years|3125.2861356932153|30.68|
|4 - 7 years|3601.6954022988507|22.41|
|< 1 year|2952.453488372093|40.70|
|> 7 years|3224.624505928854|25.30|
|unemployed|4216.758064516129|37.10|
+-----+-----+-----+

[ec2-user@ip-172-31-84-19 scripts]$
```

Task 5: Machine Learning with AWS SageMaker Autopilot :

Amazon SageMaker Autopilot is a no-code automated machine learning (AutoML) tool that enables users to build, train, and deploy machine learning models with minimal effort. It automatically explores different combinations of algorithms and data preprocessing steps to identify the best-performing model. We need to provide the dataset stored in an S3 bucket or local computer, specify the target column (dependent variable), and identify the independent variables (features) for the model.

SageMaker Autopilot automatically performs data preprocessing, feature engineering, model selection, hyperparameter tuning, and model evaluation. It generates multiple candidate models, ranks them based on their performance, and provides insights into how each model performs. The best model can then be deployed to an endpoint for real-time or batch predictions. This process allows users to build robust ML models without requiring extensive coding or deep machine learning knowledge.

Steps:

Importing Processed Data:

In previous versions, there was an option to import data directly from S3, but this feature seems to be missing in the current version. As a result, I uploaded the dataset from my local machine instead. However, I still utilized S3 by downloading the preprocessed file from it. This approach allowed me to leverage the preprocessed data for further analysis and modeling, ensuring efficiency and consistency in the workflow.

The image shows that the credit file has been successfully imported and is ready to be used for building an ML model using AutoML.

The screenshot displays the AWS SageMaker Autopilot console interface. The top navigation bar shows 'My models > loan_default_model > Version 1'. The main content area is titled 'Select dataset' and includes a search bar for datasets in Canvas. A table lists available datasets, with the 'credit' dataset selected as the target variable (Y1). The table columns are Name, Columns, Rows, Cells, Created, and Status.

Name	Columns	Rows	Cells	Created	Status	
credit	Y1	18	1,000	18,000	12/08/2024 4:54 PM	Ready
canvas-sample-loans-part-2.csv	Y1	5	1,000	5,000	12/08/2024 4:47 PM	Ready
canvas-sample-shipping-logs.csv	Y1	12	1,000	12,000	12/08/2024 4:47 PM	Ready
canvas-sample-databricks-dolly-15k.csv	Y1	2	10,879	21,758	12/08/2024 4:47 PM	Ready
canvas-sample-retail-electronics-forecasting.csv	Y1	6	40,500	243,000	12/08/2024 4:47 PM	Ready
canvas-sample-diabetic-readmission.csv	Y1	16	1,000	16,000	12/08/2024 4:47 PM	Ready
canvas-sample-housing.csv	Y1	10	1,000	10,000	12/08/2024 4:47 PM	Ready
canvas-sample-product-descriptions.csv	Y1	5	120	600	12/08/2024 4:47 PM	Ready
canvas-sample-maintenance.csv	Y1	9	1,000	9,000	12/08/2024 4:47 PM	Ready
canvas-sample-loans-part-1.csv	Y1	19	1,000	19,000	12/08/2024 4:47 PM	Ready

Run Autopilot Experiment :

For this analysis, the target column is **default**, which indicates whether a borrower defaults on a loan or not. This column contains binary values — **yes** for default and **no** for non-default. Identifying the target column is crucial as it defines the objective of the machine learning model, which in this case is to predict the likelihood of loan default.

The remaining columns, such as **checking_balance**, **credit_history**, **employment_duration**, **age**, and **purpose**, act as independent features. These features provide information related to the borrower's financial status, creditworthiness, and demographic details, which are used by the model to predict the target variable. By selecting **default** as the target, AWS SageMaker AutoML can train and evaluate various machine learning models to identify the patterns that influence loan defaults.

The image below shows the selection of the **target column (default)** along with the relevant **features (independent variables)** required for model training.

My models > loan_default_model > Version 1

Build tab: Select a column to predict. Target column: default. Value distribution: no, yes.

Model type
SageMaker Canvas automatically recommends the appropriate model type for your analysis.
2 category prediction
Your model classifies default into two categories.
Configure model

credit
Full dataset: 1.0k rows

Column name	Data type	Feature type	Missing	Mismatched	Unique	Mode
years_at_residence	123 Numeric	-	0.00% (0)	0.00% (0)	4	4
savings_balance	Text	Categorical	0.00% (0)	0.00% (0)	5	< 100 DM
purpose	Text	Categorical	0.00% (0)	0.00% (0)	6	furniture/appliances
phone	Text	Binary	0.00% (0)	0.00% (0)	2	no
percent_of_income	Text	-	0.00% (0)	0.00% (0)	4	4
other_credit	Text	Categorical	0.00% (0)	0.00% (0)	1	none
months_loan_duration	123 Numeric	-	0.00% (0)	0.00% (0)	33	24
loan_duration_category	Text	Categorical	0.00% (0)	0.00% (0)	3	Medium-Term
job	Text	Categorical	0.00% (0)	0.00% (0)	4	skilled
housing	Text	Categorical	0.00% (0)	0.00% (0)	3	own
existing_loans_count	123 Numeric	-	0.00% (0)	0.00% (0)	4	1
employment_duration	Text	Categorical	0.00% (0)	0.00% (0)	5	1 - 4 years

Total columns: 15 | Total rows: 1,000 | Total cells: 10,000 | Show dropped columns

The AutoML process was successfully executed, training and evaluating multiple machine learning models. SageMaker Autopilot automatically explored various algorithms and hyperparameter combinations to identify the most accurate model for predicting loan defaults. The process provided detailed insights into model performance, including accuracy metrics and the best-performing model selection.

SageMaker Canvas automatically selected a binary classification model for predicting the "default" variable, which has two categories: "yes" and "no." This model type was chosen based on the nature of the data and the classification task.

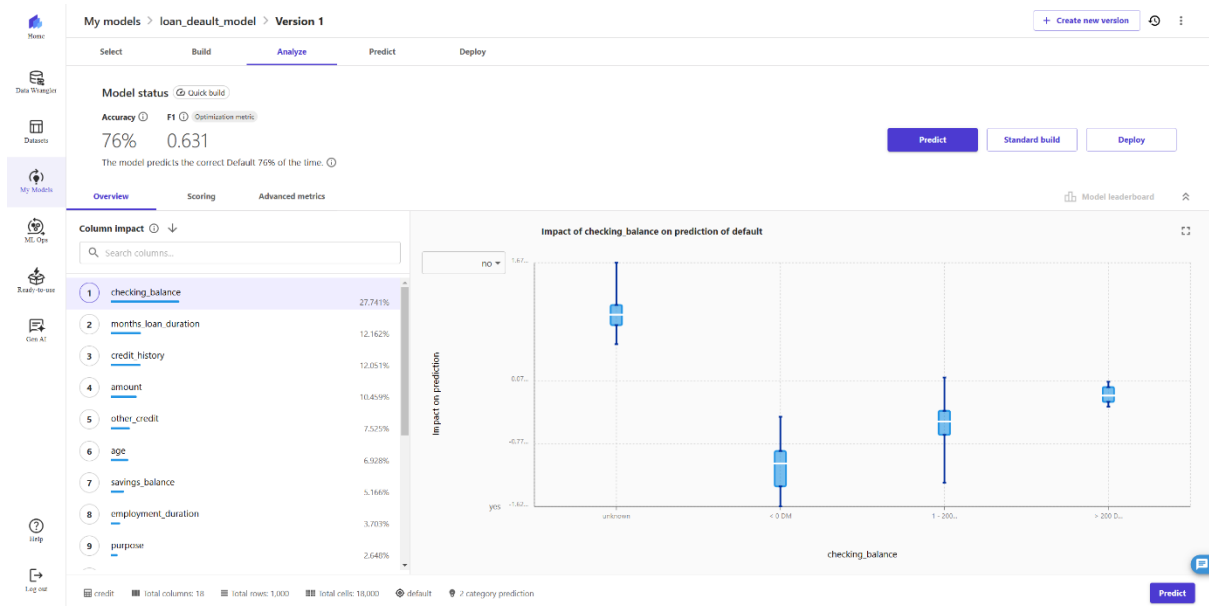
Model type

SageMaker Canvas automatically recommends the appropriate model type for your analysis.

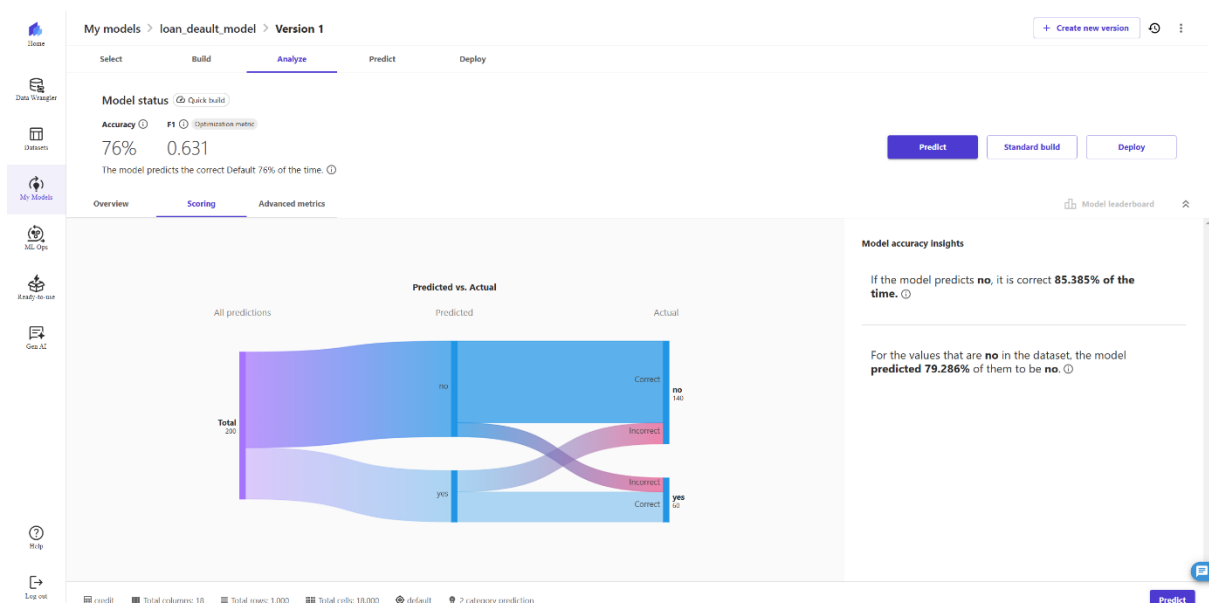
2 category prediction

Your model classifies default into two categories.

[Configure model](#)



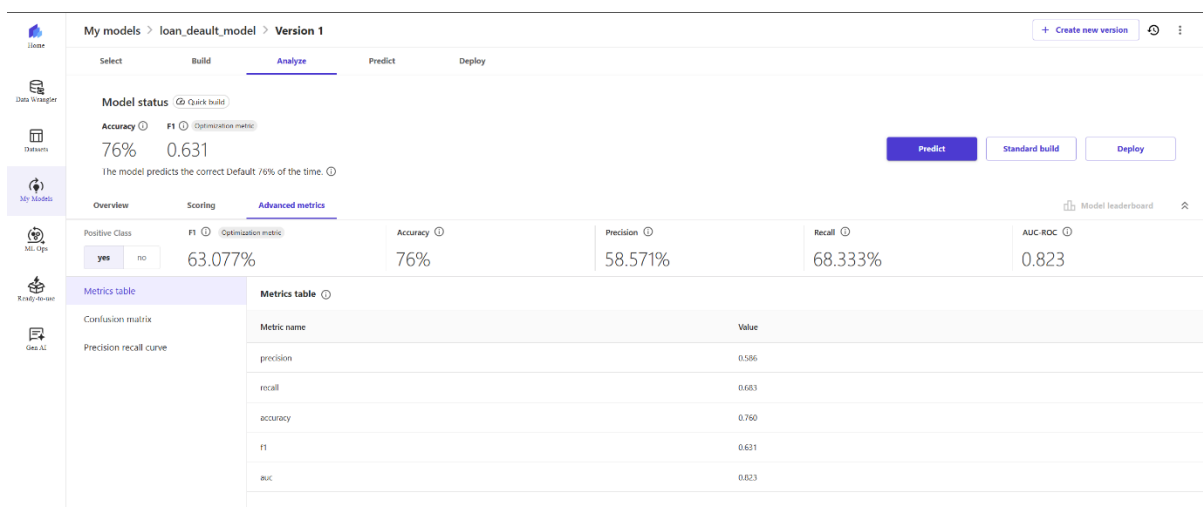
The image above highlights the successful training of the model, achieving an accuracy of 76% and an F1 score of 0.631. These metrics indicate a balanced trade-off between precision and recall, reflecting the model's ability to correctly predict loan defaults. Additionally, the window provides a detailed analysis of feature importance, showcasing the relative impact each feature has on the target variable. Visual plots are also displayed, offering a clear, graphical representation of feature contributions, which aids in understanding how specific attributes influence the model's predictions. This comprehensive analysis supports better interpretability and enhances decision-making.



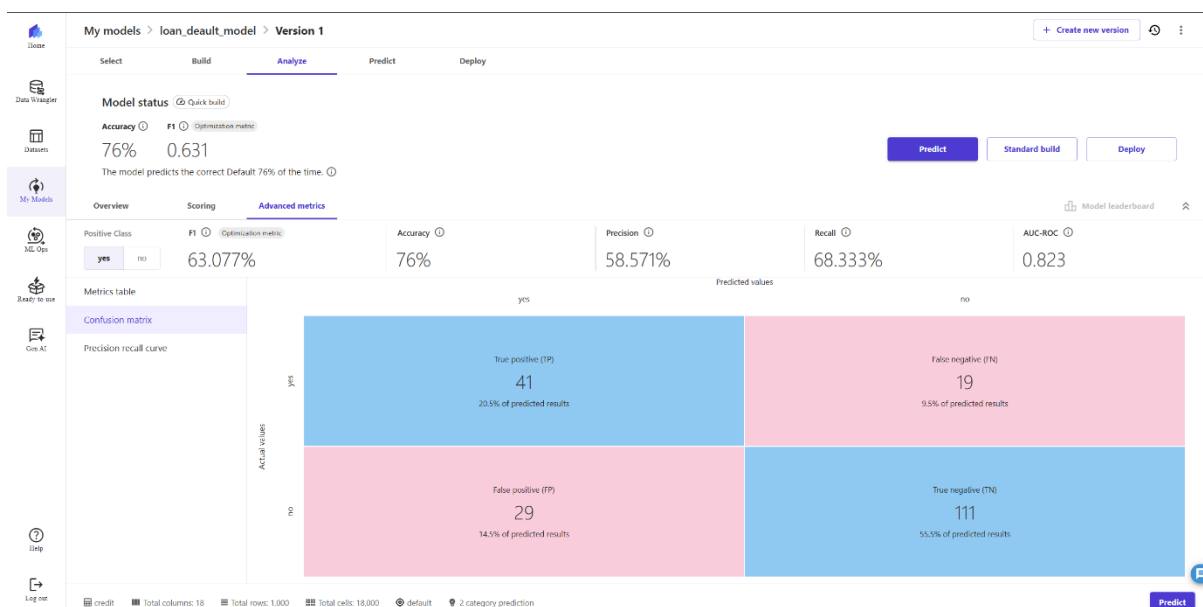
The model's performance evaluation reveals key insights into its predictive capabilities. When the model predicts "no" for the target variable, it is correct 85.39% of the time, indicating strong precision for this category. Additionally, for all the actual "no" values in the dataset, the model successfully

identifies 79.29% of them as "no," reflecting its recall for this class. These metrics demonstrate the model's effectiveness in minimizing false positives and capturing the majority of true "no" instances, offering a well-rounded perspective on its reliability and accuracy. The scoring section, illustrated with a band-style visualization, provides a clear visual representation of these performance metrics.

In the image below, we can observe the performance metrics of the trained model, which include an F1 score of 63.07%, accuracy of 76%, precision of 58.57%, recall of 68.33%, and an AUC-ROC of 0.823. These metrics provide a comprehensive evaluation of the model's effectiveness. The F1 score, balancing precision and recall, indicates moderate performance in classifying both positive and negative instances. The accuracy shows that the model correctly predicts the target variable 76% of the time. Precision and recall highlight that while the model identifies a significant proportion of actual positives, there's room for improvement in minimizing false positives. The AUC-ROC score of 0.823 indicates that the model has good discriminatory power, distinguishing between the positive and negative classes effectively.



In the image, we can see the confusion matrix for the trained model, which shows the following results: 41 True Positives (TP), 19 False Negatives (FN), 29 False Positives (FP), and 111 True Negatives (TN). These values give a detailed picture of the model's classification performance. The 41 TP indicates the number of times the model correctly predicted the positive class, while the 111 TN shows the correct predictions for the negative class. The 19 FN represents cases where the model failed to predict the positive class, and the 29 FP indicates instances where the model incorrectly predicted the positive class. Analyzing these values helps in understanding the balance between precision, recall, and the overall performance of the model.



Model Analysis and Performance Metrics

The model built using SageMaker Canvas was evaluated on several performance metrics, including **Accuracy (76%)**, **F1 Score (63.07%)**, **Precision (58.57%)**, **Recall (68.33%)**, and **AUC-ROC (0.823)**. The **confusion matrix** revealed 41 true positives (TP), 19 false negatives (FN), 29 false positives (FP), and 111 true negatives (TN). These metrics indicate that the model performs reasonably well, with a moderate balance between precision and recall. The AUC-ROC score of 0.823 suggests that the model has a good ability to distinguish between the two classes, "yes" and "no," for the default prediction.

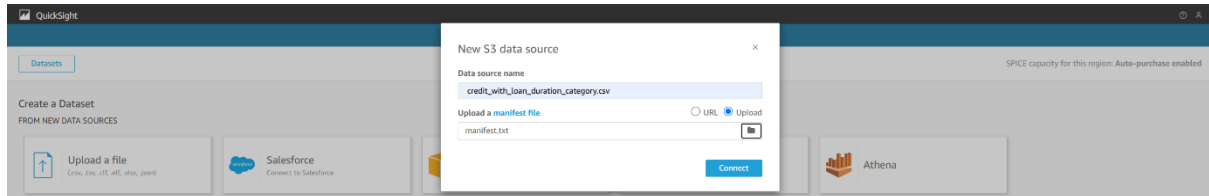
Ethical Considerations: Bias and Privacy

1. **Bias in Training Data:** One of the potential ethical concerns when building models like this is **bias in training data**. The dataset includes features such as age, employment duration, credit history, and housing status, which could inadvertently reflect societal biases. For example, if certain demographic groups are underrepresented or overrepresented in the data, the model might make biased predictions that favor one group over another. This could lead to discriminatory outcomes, particularly in sensitive areas like loan approval, defaulter prediction and credit scoring.
 - **Addressing Bias:** It's important to assess the fairness of the model by analyzing its performance across different demographic groups. Techniques such as **fairness-aware modeling** or post-processing the model's predictions can help mitigate bias. Additionally, diversifying the data sources and ensuring that various groups are represented in the training data can reduce bias in predictions.
2. **Privacy Concerns:** The dataset includes personal information such as credit history, income, and employment duration, which are highly sensitive. There is a risk that personal data could be exposed or misused during the model training and evaluation process. In particular, improper handling of such data could lead to **violations of privacy** and misuse by unauthorized parties.
 - **Addressing Privacy:** To address privacy concerns, it is essential to ensure **data anonymization** and **secure data storage**. Using privacy-preserving techniques, such as **differential privacy** or **encryption**, during data processing can safeguard personal information. Additionally, ensuring that only authorized personnel have access to the data and following data protection regulations (such as GDPR) are key steps to uphold privacy standards.

In conclusion, while the model demonstrates strong performance based on standard evaluation metrics, it is crucial to be aware of potential ethical issues such as **bias in the data** and **privacy risks**. Continuous monitoring and mitigation strategies must be employed to ensure the model's fairness and protect users' privacy throughout its lifecycle.

4. Visualization:

AWS QuickSight was used to generate insightful visualizations for the analysis. To begin, an AWS QuickSight account was created, providing access to the platform's powerful data visualization tools. The account was then configured to connect to the S3 as one of the primary data sources, allowing the seamless creation of various charts and dashboards to explore patterns and relationships in the dataset. Through AWS QuickSight, key metrics were visualized effectively, aiding in a deeper understanding of the data and its underlying trends.



When creating an account in QuickSight, several options are available for selecting various AWS sources that can serve as the input data for visualizations S3 buckets were selected here. As shown in the image above, to connect QuickSight to S3 and import the desired file, we need to provide the file name along with a manifest file. The manifest file contains metadata about the S3 location and the structure of the dataset, allowing QuickSight to correctly access and process the data stored in S3.

Visualizations:

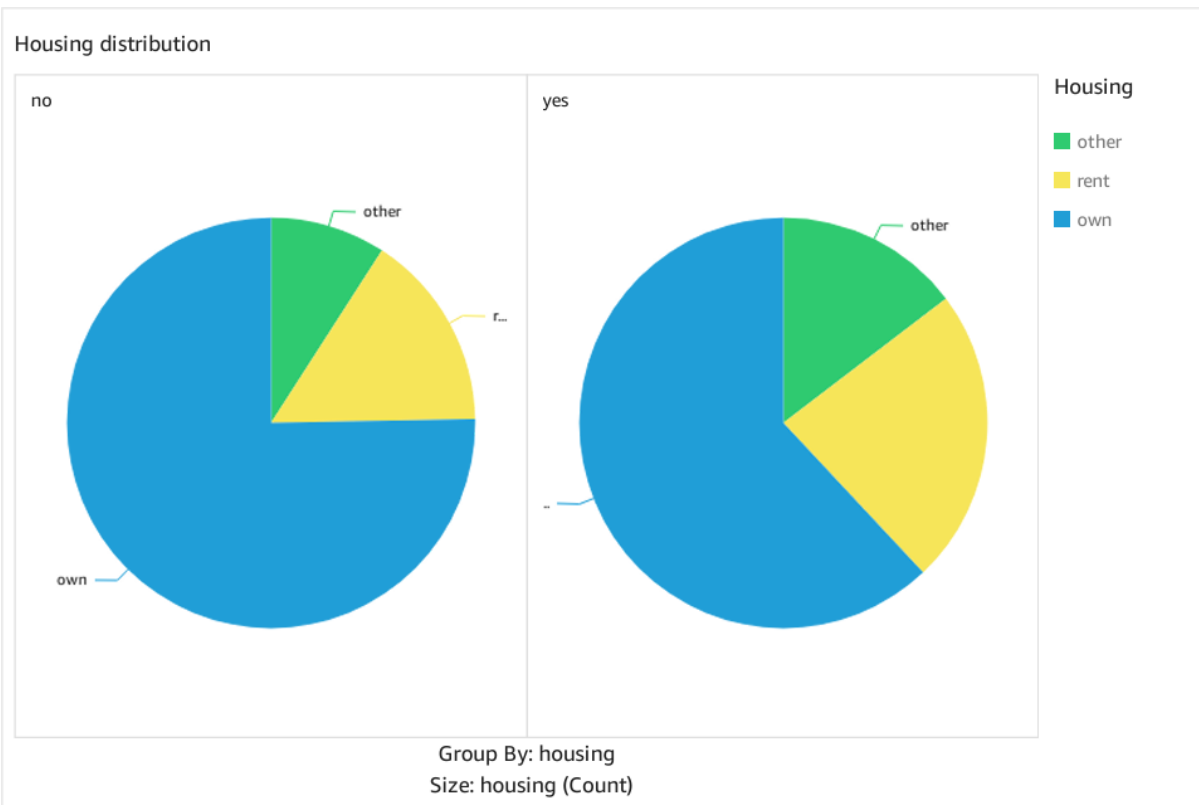
1. Distribution of Housing Types by Loan Default Status

Type of Plot: Pie Chart

In the visualization, two pie charts are displayed side by side, one for "Yes" (loan default) and the other for "No" (no loan default). Each pie chart represents the distribution of housing types (rent, own, and other) for the respective group.

- The first pie chart visualizes the distribution of housing types for customers who have defaulted on their loan (marked as "Yes").
- The second pie chart shows the distribution of housing types for customers who have not defaulted on their loan (marked as "No").

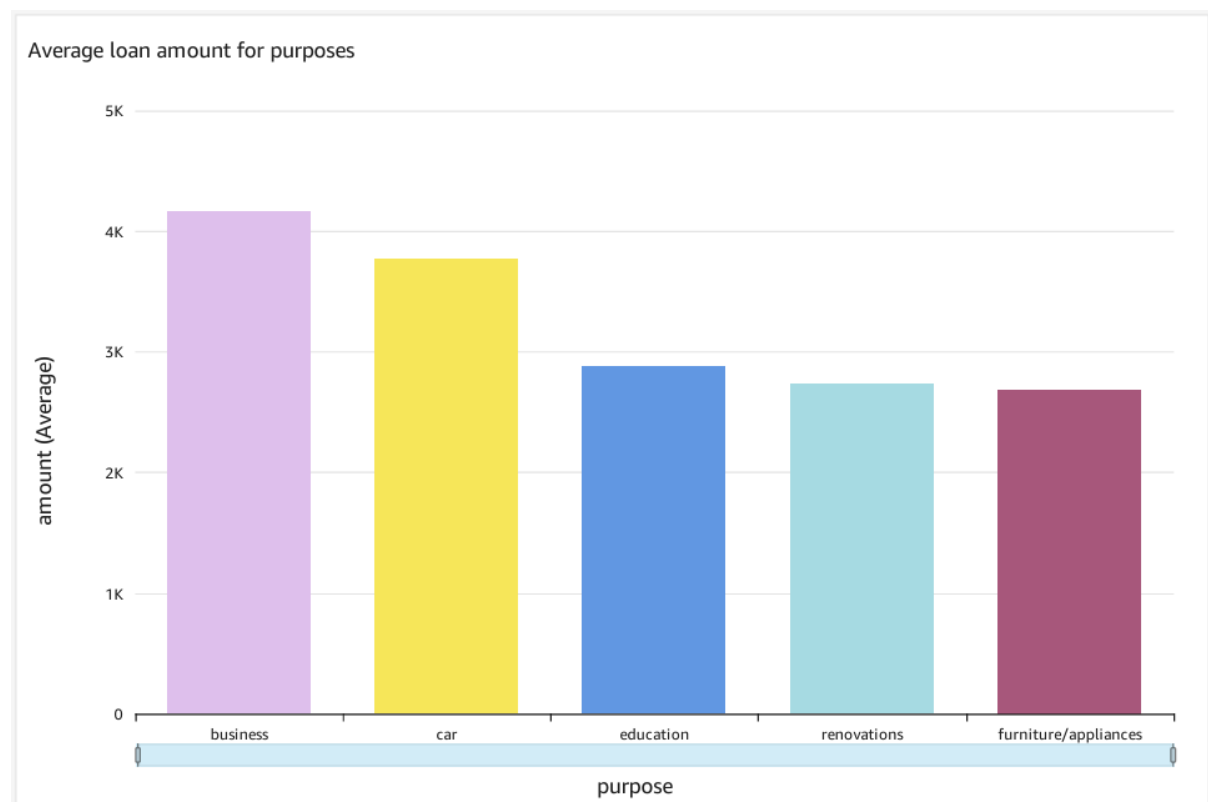
By comparing these two pie charts, we can analyze whether the type of housing (rent, own, or other) has any noticeable correlation with the likelihood of loan default. This can help identify trends or patterns that may influence the probability of default based on the customer's housing status.



2. Average Loan Amount by Purpose

Type of Plot: Bar Chart

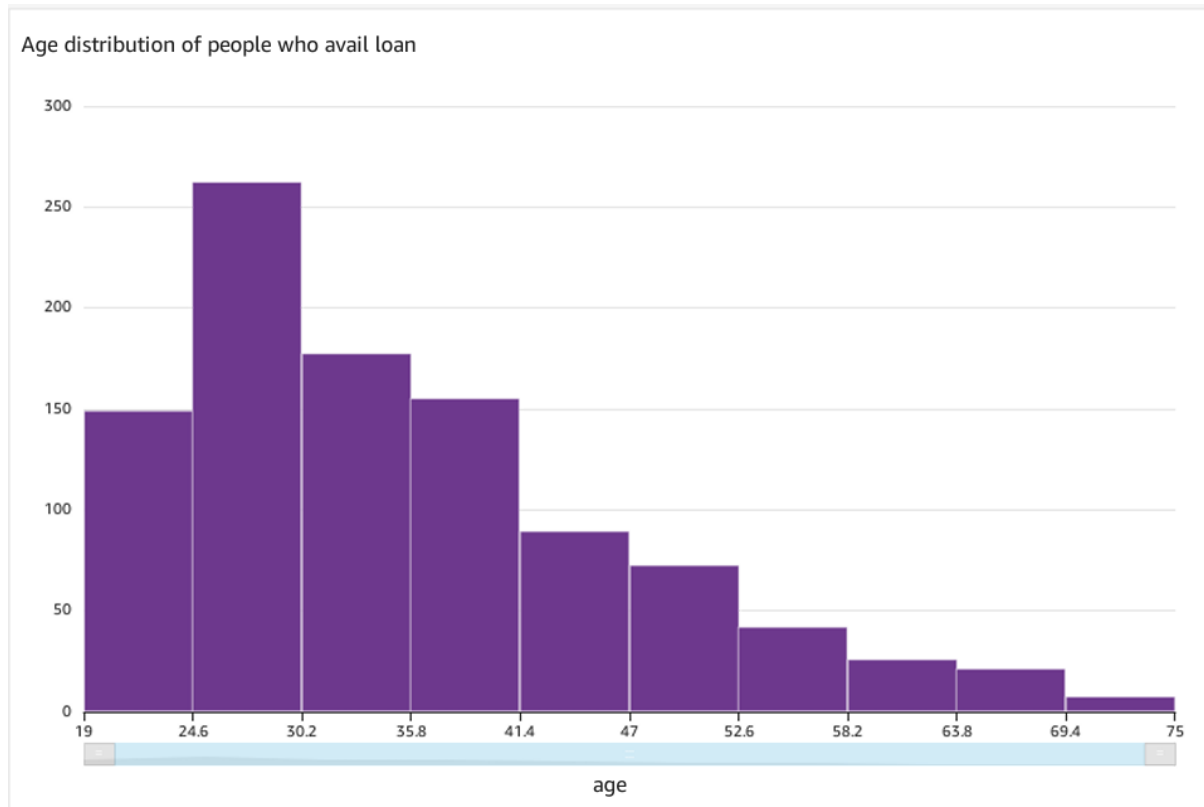
This bar chart displays the average loan amounts categorized by the purpose of the loan, such as furniture/appliances, education, or car purchase. It provides a clear comparison of how loan amounts vary depending on the intended use of the loan. The chart helps to visualize which purposes are associated with higher or lower average loan amounts, aiding in understanding spending patterns.



3. Age Distribution of Applicants

Type of Plot: Histogram

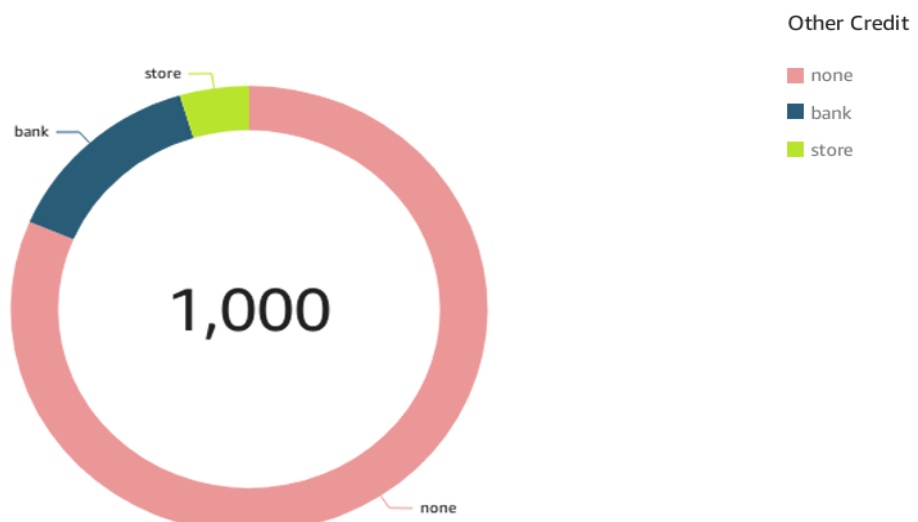
This histogram illustrates the age distribution of all individuals in the dataset. It shows how many applicants fall within specific age ranges, providing insight into the age demographics of the dataset. The chart highlights trends such as whether younger or older applicants are more likely to apply for loans, which can inform decisions or reveal patterns in loan application behavior.



4. Distribution of Other Credit or Assets Owned

Type of Plot: Doughnut Chart

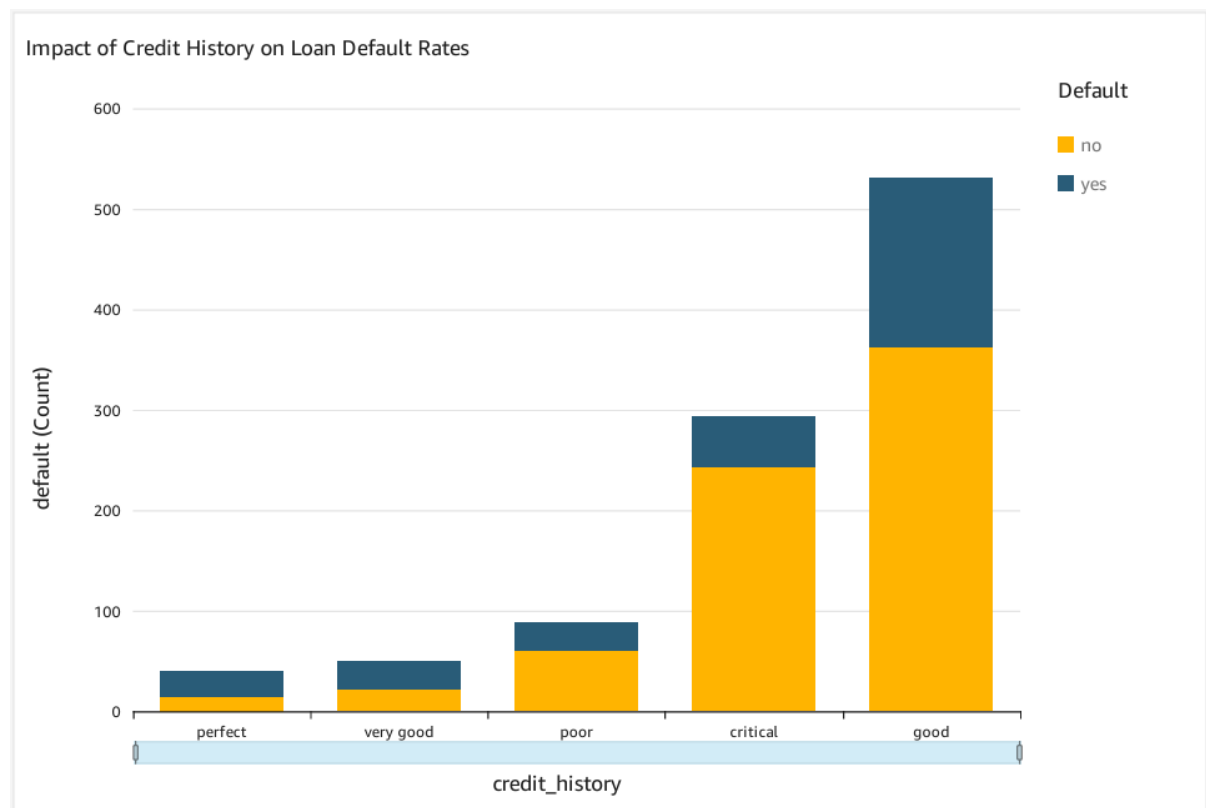
The doughnut chart visualizes the distribution of "other credit" or assets owned by the individuals in the dataset. It provides a clear view of the proportion of applicants who own different types of credit or assets, such as "none," "real estate," or "life insurance." This chart helps in understanding how ownership of additional assets correlates with applicants' financial status and might be used to assess their creditworthiness or stability.



5. Credit History vs Default Count

Type of Plot: Vertical Stacked Bar Chart

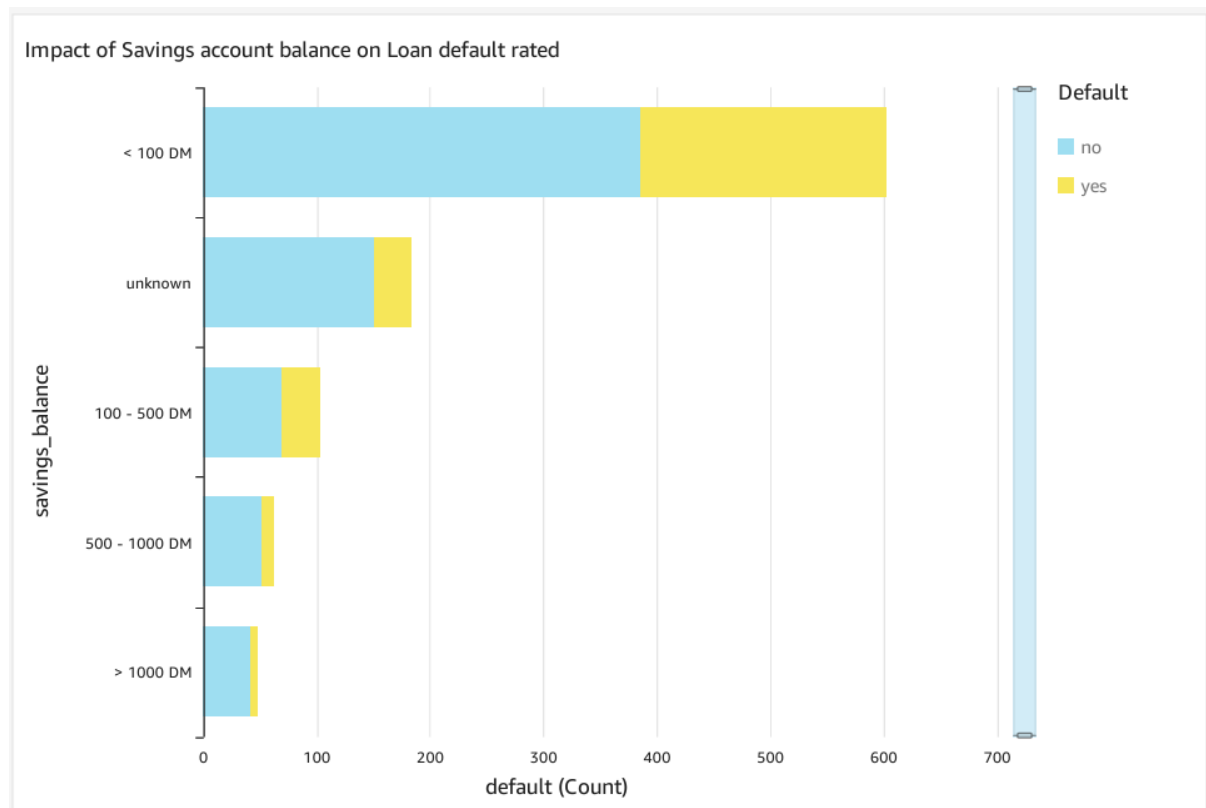
The vertical stacked bar chart displays the relationship between "credit history" and the count of loan defaults. On the X-axis, it categorizes applicants based on their credit history, such as "good," "critical," or "poor." The Y-axis represents the number of defaults within each credit history category. The stacked bars indicate how the defaults are distributed across different credit history types, providing insights into how varying credit histories affect the likelihood of default. This visualization can be useful in understanding the risk associated with different credit history categories.



6. Savings Account vs Default Count

Type of Plot: Horizontal Stacked Bar Chart

The horizontal stacked bar chart illustrates the relationship between "savings account" categories and the default count. The Y-axis categorizes applicants based on their savings account status, such as "unknown," "< 100 DM," or ">= 100 DM." The X-axis represents the number of defaults within each savings account category. The stacked bars show how defaults are distributed across these savings account categories, allowing us to assess whether a specific savings account balance correlates with a higher or lower likelihood of default. This visualization provides valuable insights into how savings account status impacts the likelihood of loan default.



Summary of Visualizations and Experience with AWS QuickSight

In this analysis, I created a series of visualizations using AWS QuickSight to explore various aspects of loan data and default rates. The visualizations included:

1. **Pie Charts for Default ("Yes" and "No") vs Housing Type:** These charts illustrated the distribution of loan defaults based on housing status, highlighting how the likelihood of default varied across those with "own," "rent," and "other" housing types.
2. **Bar Chart of Average Loan Amounts for Different Purposes:** This chart displayed the average loan amount across various loan purposes, helping to understand trends in borrowing behavior for categories like "furniture/appliances" and "education."
3. **Histogram for Age Distribution:** The histogram revealed the age distribution of applicants, which is essential for assessing the demographic makeup of the dataset.
4. **Doughnut Chart for Other Credit Types:** This chart provided insights into the distribution of other credits or assets owned by the applicants, helping to assess the financial diversity of the dataset.
5. **Vertical Stacked Bar Chart for Credit History vs Default Count:** This chart visualized the relationship between credit history and default counts, revealing how different credit histories impacted the likelihood of default.
6. **Horizontal Stacked Bar Chart for Savings Account vs Default Count:** This chart examined the connection between savings account status and default rates, highlighting the financial stability of applicants in relation to default behavior.

Experience with AWS QuickSight

Using AWS QuickSight to generate these visualizations was an enriching experience. The interface was intuitive, and the steps for connecting data sources, preparing datasets, and creating visualizations were straightforward. However, I faced a few challenges along the way, such as:

- **Connecting the S3 data source:** Initially, I encountered an issue with linking the S3 bucket correctly. After ensuring that the manifest file was properly configured and pointing to the correct endpoint, the issue was resolved, and I was able to load the data seamlessly.
- **Data formatting for complex visualizations:** While setting up the complex stacked bar charts, I faced difficulties in ensuring that the data was grouped and stacked properly. After reviewing the dataset and making sure that the fields were correctly aggregated and categorized, the charts displayed as expected.

Despite these initial hurdles, QuickSight's user-friendly interface and helpful documentation made it easier to overcome these challenges. Overall, the process of creating these insightful visualizations was smooth and efficient, offering a clear view of the dataset and valuable insights into loan defaults.